# Vox: Technical Architecture Specification

Text-to-Speech Engine for AI Agents

March 2026 — v1.4.0

## Contents


**1 Overview** **2**
- 1.1 Design Principles . . . 2
- 1.2 Component Map . . . 2

**2 System Architecture** **3**
- 2.1 Daemon and Connectivity . . . 3
- 2.2 System Paths . . . 3
- 2.3 Service Install and Identity . . . 4
- 2.4 Data Paths . . . 4
- 2.5 TTS Request Path (Text → Audio File → Playback) . . . 4
- 2.6 Notification Path (Hook Event → Audio) . . . 4
- 2.7 Session Lifecycle . . . 5

**3 Provider Architecture** **5**
- 3.1 Provider Protocol . . . 5
- 3.2 ElevenLabs . . . 5
- 3.3 OpenAI . . . 6
- 3.4 AWS Polly . . . 6
- 3.5 Offline Fallbacks . . . 6
- 3.6 Provider Selection . . . 6

**4 MCP Tool Surface** **7**
- 4.1 Background Synthesis Pattern . . . 7
- 4.2 Transport . . . 7

**5 Audio Pipeline** **7**
- 5.1 Text Splitting . . . 7
- 5.2 Segment Processing . . . 8
- 5.3 Audio Stitching . . . 8
- 5.4 Playback Queue . . . 8
- 5.5 Ephemeral Mode . . . 9
- 5.6 Format Handling . . . 9

**6 CLI and Config** **9**
- 6.1 CLI Commands . . . 9
- 6.2 Config File Format . . . 9
- 6.3 Credential Management . . . 10

**7 Performance** **10**
- 7.1 Latency Characteristics . . . 10
- 7.2 Caching . . . 10
- 7.3 Streaming vs. Batch . . . 11

**8 Security** **11**
8.1 Threat Model . . . 11
8.2 No Audio Until Opt-In . . . 11
8.3 Credential Isolation . . . 11

**9 Platform Integration** **12**
9.1 macOS . . . 12
9.2 Linux . . . 12

**10 Plugin and Hook Integration** **12**
10.1 Hook Registration . . . 12
10.2 Mute/Unmute Lifecycle . . . 13
10.3 Signal Classification . . . 13
10.4 Mood Inference . . . 14

**11 Known Limitations** **14**
11.1 No Streaming Playback . . . 14
11.2 No Intentional Audio Caching . . . 14
11.3 No Provider Failover . . . 14
11.4 No Playback Cancellation . . . 14
11.5 Background Failure Opacity . . . 14
11.6 Concurrent Config Writes . . . 14
11.7 NFS File Locking . . . 15
11.8 No Headless Audio Detection . . . 15
11.9 Voice List Staleness . . . 15

**12 Test Architecture** **15**

# 1 Overview

Vox is a text-to-speech engine with three projection surfaces from a single codebase: Python CLI, MCP server, and Claude Code plugin. It supports five synthesis backends (ElevenLabs, OpenAI, AWS Polly, macOS `say`, Linux `espeak-ng`) behind a provider-agnostic orchestration layer. Audio playback is serialized via file locking. Session state lives in a YAML frontmatter config file.

The system does *not* stream audio. Every synthesis call produces a complete MP3 file before playback begins. There is no audio mixing and no real-time pitch manipulation. Vox is a batch synthesizer with a playback queue. The `voxd` daemon accepts requests over WebSocket and serializes playback.

## 1.1 Design Principles

**P1**. **Provider-agnostic orchestration.** The core module knows nothing about any specific TTS API. Providers implement a `TTSProvider` protocol; the core synthesizes, splits, stitches, and plays without knowing which backend produced the audio.

**P2**. **No audio until opt-in.** Vox is silent by default. The `notify` config field defaults to `"n"`. No hooks produce sound, no chimes play, and no synthesis calls fire until the user explicitly enables notifications via `/unmute` or `/vox`.

**P3**. **Session-scoped state.** Mutable state (voice, provider, mood) lives in the MCP server's memory, one instance per Claude Code session. Initial state is seeded from `.vox/config.md` at startup; hook handlers also read and write this file for signal accumulation. The `voxd` daemon is stateless with respect to sessions—it synthesizes and plays whatever clients send it.

**P4**. **Fail fast.** Invalid input raises exceptions. No defensive fallbacks, no silent degradation. If a voice doesn't exist, the call fails with a `VoiceNotFoundError`—it does not silently substitute another voice.

**P5**. **Serialized playback.** Concurrent synthesis calls never overlap audio output. The `voxd` daemon owns an in-memory playback queue with a single consumer—audio items play sequentially across all connected clients.

## 1.2 Component Map

| Module | File | Responsibility |
|---|---|---|
| Types | `src/punt_vox/types.py` | Domain types: `TTSProvider` protocol, `AudioRequest`, `AudioResult`, `HealthCheck`, `MergeStrategy` enum |
| Core | `src/punt_vox/core.py` | `TTSClient`—provider-agnostic orchestration: text splitting, batch synthesis, pair stitching, audio merge |
| Config | `src/punt_vox/config.py` | Read/write `.vox/config.md` YAML frontmatter; worktree-safe path resolution |
| Resolve | `src/punt_vox/resolve.py` | Voice/language resolution, output directory resolution, vibe tag application |
| Output | `src/punt_vox/output.py` | Output path resolution: `VOX_OUTPUT_DIR` env, `~/vox-output` fallback |
| Playback | `src/punt_vox/playback.py` | `flock`-serialized playback: `play_audio()` (blocking), `enqueue()` (detached subprocess) |
| Hooks | `src/punt_vox/hooks.py` | Claude Code hook dispatchers: Stop, PostToolUse, Notification, PreCompact, UserPromptSubmit, SubagentStart/Stop, SessionEnd |
| Voices | `src/punt_vox/voices.py` | Voice personality blurbs for roster display |
| Quips | `src/punt_vox/quips.py` | Immutable phrase pools for hook speech (stop, permission, idle, farewell, etc.) |

| Module | File | Responsibility |
|---|---|---|
| Mood | `src/punt_vox/mood.py` | Mood family classification (bright/dark/neutral) |
| Logging | `src/punt_vox/logging_config.py` | Rotating file log to `~/.punt-labs/vox/logs/tts.log` |
| Watcher | `src/punt_vox/watcher.py` | `SessionWatcher`—daemon thread tailing Claude session JSONL files for real-time signal classification and autonomous chime/speech in continuous mode |
| Daemon | `src/punt_vox/voxd.py` | Audio daemon (`voxd`): WebSocket server for synthesize, chime, record, voices, health. Playback queue, dedup, cache. No MCP, no sessions |
| Client | `src/punt_vox/client.py` | WebSocket client for `voxd`. `VoxClient` (async), `VoxClientSync` (sync). Lightweight—stdlib + websockets only |
| Providers | `src/punt_vox/providers/` | Five backends: ElevenLabs, OpenAI, Polly, `say`, `espeak-ng` |
| Server | `src/punt_vox/server.py` | FastMCP server (key: `mic`): `unmute`, `record`, `vibe`, `who`, `notify`, `speak`, `status` |
| CLI | `src/punt_vox/__main__.py` | Typer CLI: unmute, record, vibe, notify, speak, voice, status, doctor, version, hook subcommands |

# 2 System Architecture

## 2.1 Daemon and Connectivity

The system has two binaries from one package (`punt-vox`):

`voxd` System-level audio daemon. Listens on port 8421 via WebSocket. Owns synthesis (providers), playback queue, audio deduplication, and synthesis cache. Knows nothing about MCP, hooks, projects, or Claude Code.

`vox` Everything else: CLI commands, MCP server (`vox mcp`), hook handlers (`vox hook <event>`). All are WebSocket clients of `voxd`.

```
Claude Code <-- stdio --> vox mcp -- WebSocket --> voxd :8421
Hook scripts --> vox hook <event> -- WebSocket -->    |
Shell        --> vox unmute "hi"  -- WebSocket -->    |
                                                       v
                                                   speakers
```

The MCP session (Claude Code ↔ `vox mcp`) is stdio—unaffected by daemon restarts. The WebSocket connection (`vox mcp` ↔ `voxd`) reconnects automatically.

## 2.2 System Paths

`voxd` stores runtime state in system directories. The synthesis cache is in the user's home directory (`~/.punt-labs/vox/cache/`).

| Purpose | macOS (Homebrew) | Linux |
|---|---|---|
| Config | `$(brew --prefix)/etc/vox/keys.env` | `/etc/vox/keys.env` |
| Cache | `~/.punt-labs/vox/cache/` | |
| Logs | `$(brew --prefix)/var/log/vox/voxd.log` | `/var/log/vox/voxd.log` |
| Runtime | `$(brew --prefix)/var/run/vox/serve.{port,token}` | `/var/run/vox/serve.{port,token}` |
| Service | `/Library/LaunchDaemons/com.punt-labs.voxd.plist` | `/etc/systemd/system/voxd.service` |

Per-project enablement (`.vox/config.md`) is a client-side file in the project directory. The daemon never reads it.

## 2.3 Service Install and Identity

```
sudo vox daemon install
```

Requires `sudo` because the service plist (macOS) or unit (Linux) is written to a system directory. `voxd` runs as the *installing user*, not root—it needs audio device access tied to the desktop session. The LaunchDaemon plist sets `UserName`; the systemd unit sets `User=`.

On macOS, `voxd` detects the Homebrew prefix at startup via `brew --prefix` (default `/opt/homebrew` on Apple Silicon, `/usr/local` on Intel).

## 2.4 Data Paths

Three primary data paths define the system's runtime behavior.

## 2.5 TTS Request Path (Text → Audio File → Playback)

1. Agent calls MCP tool (e.g., `unmute(text="Hello")`).
2. Server resolves provider: explicit parameter → session config → auto-detection (environment variable → API key probe → platform fallback).
3. Server resolves voice: explicit parameter → session config → language-aware default → provider default.
4. Server builds `AudioRequest` list (one per segment).
5. Server predicts output paths and returns immediately to the agent with predicted `AudioResult` metadata.
6. Background daemon thread: `TTSClient.synthesize_batch()` calls the provider's `synthesize()` for each request.
7. If multiple segments: `stitch_audio()` concatenates MP3 files with configurable inter-segment silence (default 500 ms) plus 150 ms trailing silence.
8. `enqueue(path)` copies the file to `~/.punt-labs/vox/pending/`, spawns a detached subprocess.
9. Subprocess acquires `flock(LOCK_EX)` on `~/.punt-labs/vox/playback.lock`, plays via `afplay` (macOS) or `ffplay` (cross-platform), deletes the pending copy on completion.

The agent receives the MCP response at step 5. Steps 6–9 happen asynchronously. The agent never blocks on synthesis or playback.

## 2.6 Notification Path (Hook Event → Audio)

1. Claude Code fires a lifecycle hook (Stop, Notification, PreCompact, etc.).
2. Shell script dispatcher (e.g., `hooks/notify.sh`) invokes `vox hook stop` with JSON on stdin.
3. Python hook handler reads session config, checks `notify` and `speak` fields.
4. If `notify` is `"n"`: no-op, immediate return.
5. If `speak` is `"n"` (chime mode): resolves mood-aware chime file, enqueues playback.
6. If `speak` is `"y"` (voice mode) and `vibe_signals` is non-empty: selects a random phrase from the relevant quip pool, calls `vox unmute` to synthesize and play. If `vibe_signals` is empty (no bash commands have run yet), the Stop hook is a no-op even with voice enabled.
7. For Stop hooks: returns {`"decision": "block", "reason": phrase`} to halt Claude's output while audio plays.

### 2.7 Session Lifecycle

1. **Session start.** `SessionStart` hook runs `session-start.sh`: deploys slash commands, cleans retired commands, auto-allows MCP tools. No audio plays.
2. **User enables voice.** `/unmute` or `/vox` sets `notify` to `"y"` or `"c"`. This is the first moment audio becomes possible.
3. **Signal accumulation.** Each `PostToolUse` (Bash) event classifies stdout against regex patterns and appends a timestamped signal (e.g., `tests-pass@10:45`) to `vibe_signals` in config.
4. **Mood inference.** On Stop hooks, the signal history is deterministically mapped to ElevenLabs expressive tags (e.g., `[relieved] [satisfied]`) without any LLM call.
5. **Session end.** `SessionEnd` hook speaks a farewell phrase and clears `vibe_signals`.

## 3 Provider Architecture

### 3.1 Provider Protocol

All providers implement `TTSProvider`, a Python `Protocol` class extending `AudioProvider`. Eleven required members:

| Member | Signature and Purpose |
|---|---|
| `name` | `str` — Short identifier (e.g., `"elevenlabs"`) |
| `default_voice` | `str` — Fallback voice name |
| `supports_expressive_tags` | `bool` — Whether the provider interprets `[bracketed]` tags in text |
| `synthesize` | `(request, output_path) -> AudioResult` — Core synthesis |
| `resolve_voice` | `(name, language?) -> str` — Voice validation and normalization |
| `get_default_voice` | `(language) -> str` — Language-aware default |
| `list_voices` | `(language?) -> list[str]` — Available voices |
| `infer_language_from_voice` | `(voice) -> str \| None` — Reverse lookup (best-effort) |
| `check_health` | `() -> list[HealthCheck]` — Provider-specific diagnostics |
| `generate_audio` | `(text, voice?, ...) -> AudioResult` — Convenience wrapper (inherited from `AudioProvider`) |
| `generate_audios` | `(texts, voice?, ...) -> list[AudioResult]` — Batch convenience wrapper |

No provider SDK imports appear outside the provider's own file. Providers do import shared utilities from `core.py` (`split_text`, `stitch_audio`) for chunking, but the SDK libraries (`elevenlabs`, `openai`, `boto3`) are confined to their respective files.

### 3.2 ElevenLabs

| Parameter | Value |
|---|---|
| Default model | `eleven_flash_v2_5` (low latency, 32 languages) |
| Expressive tags | Only on `eleven_v3` |
| Output format | MP3 44.1 kHz 128 kbps |
| Char limits | 5 000–40 000 depending on model |
| Chunking | Auto-split at limit, stitch with 0 ms pause |
| Voice resolution | API fetch (cached globally), match by lowercase name or raw 20-char voice ID |
| Voice settings | stability, similarity_boost, style, speaker_boost (all optional floats 0.0–1.0) |

Streaming: the SDK returns a generator of audio chunks. Vox iterates the generator to completion and writes the full file. There is no incremental playback—the entire response is buffered to disk before `enqueue()` is called.

### 3.3 OpenAI

| Parameter | Value |
|---|---|
| Default model | `tts-1` (`tts-1-hd` available) |
| Expressive tags | Not supported |
| Max chars/call | 4 096 (hard API limit) |
| Chunking | Auto-split at 4 096 chars, stitch |
| Voices | 9 static: alloy, ash, coral, echo, fable, nova, onyx, sage, shimmer |
| Rate mapping | CLI percentage → OpenAI speed float (0.25–4.0, direct linear: rate/100) |

### 3.4 AWS Polly

| Parameter | Value |
|---|---|
| Output format | MP3 |
| Engine preference | neural > generative > long-form > standard |
| Voice resolution | API fetch (paginated `DescribeVoices`, cached) |
| Language mapping | ISO 639-1 → BCP 47 (e.g., `de` → `de-DE`) |
| Auth | AWS credential chain (`aws sts get-caller-identity`) |

Voice resolution bundles three Polly parameters (`voice_id`, `language_code`, `engine`) into a frozen `VoiceConfig` dataclass, so callers never need to know Polly's multi-parameter API.

### 3.5 Offline Fallbacks

| Provider | Platform | Notes |
|---|---|---|
| `say` | macOS | System `say` command. Default voice: `samantha`. Output: AIFF converted to MP3 via ffmpeg. No API key required. No expressive tags. |
| `espeak` | Linux | `espeak-ng` command. Voice is the language code (e.g., `en`). No API key required. No expressive tags. |

These exist so `vox doctor` and basic functionality work on machines with no cloud credentials. Audio quality is noticeably lower than cloud providers.

### 3.6 Provider Selection

Auto-detection follows a strict priority chain:

1. `TTS_PROVIDER` environment variable (explicit override).
2. `ELEVENLABS_API_KEY` present → `elevenlabs`.
3. `OPENAI_API_KEY` present → `openai`.
4. `aws sts get-caller-identity` succeeds → `polly`.
5. macOS with `say` binary → `say`.
6. Linux with `espeak-ng` or `espeak` → `espeak`.
7. Fallback: `polly` with a warning (will fail at synthesis time if no credentials).

Session-level overrides persist to `.vox/config.md` when set via MCP tools. The `get_provider()` factory checks: explicit parameter → session config provider → auto-detect. Model resolution follows the same cascade: explicit → session config (only if provider matches) → `TTS_MODEL` env → provider default.

**What this does not do:** there is no failover. If the selected provider's API call fails, the error propagates. There is no automatic retry, no fallback to a cheaper provider, and no circuit breaker.

# 4 MCP Tool Surface

The MCP server registers as key `mic` and exposes seven tools. All tools return JSON strings.

| Tool | Signature and Purpose |
|---|---|
| `unmute` | `(text?, voice?, language?, segments?, rate=90, pause_ms=500, ephemeral=true, stability?, similarity?, style?, speaker_boost?, vibe_tags?, provider?, model?)` — Synthesize and play audio. `segments` is a list of {`text, voice?, language?, vibe_tags?`} dicts for multi-voice sequential playback. Returns immediately; synthesis and playback happen in a background daemon thread. Persists provider, model, and vibe_tags to session config. |
| `record` | `(text?, voice?, language?, segments?, rate=90, pause_ms=500, output_path?, output_dir?, stability?, similarity?, style?, speaker_boost?)` — Like `unmute` but writes to persistent path (default `~/vox-output/`), no playback. |
| `vibe` | `(mood?, tags?, mode?)` — Set session mood, expressive tags, or vibe mode (`auto`/`manual`/`off`). |
| `who` | `(language?)` — List available voices: 6 random featured voices with personality blurbs, plus full roster. Filters by language if provided. |
| `notify` | `(mode?, voice?)` — Set notification mode: `y` (on-demand), `n` (off), `c` (continuous). First enable auto-sets `speak` to `"y"`. |
| `speak` | `(mode?, voice?)` — Set speech mode: `y` (voiced) or `n` (chimes only). |
| `status` | `()` — Returns full session state: provider, voice, notify, speak, vibe_mode, vibe, vibe_tags, vibe_signals. |

## 4.1 Background Synthesis Pattern

Both `unmute` and `record` return immediately with *predicted* results. The server computes deterministic output paths (MD5 hash of text → 12-hex-char filename) and returns metadata before synthesis begins. A daemon thread performs the actual provider API call and playback.

This means the agent sees the response in milliseconds, but audio may not play for several seconds (depending on provider latency). If synthesis fails in the background thread, the error is logged but not reported to the agent. The agent has no mechanism to detect synthesis failure after the tool call returns.

## 4.2 Transport

The MCP server runs on stdio transport (`stdin`/`stdout`). Logs go to stderr only. The server is started by Claude Code via `vox mcp` and communicates using the MCP JSON-RPC protocol. The MCP server delegates synthesis and playback to `voxd`, a persistent WebSocket daemon. Each Claude Code session spawns its own MCP server process, but all sessions share a single `voxd` instance.

# 5 Audio Pipeline

## 5.1 Text Splitting

`split_text(text, max_chars)` guarantees every chunk is $\leq$ `max_chars`:

1. If text fits: return as-is.
2. Split at sentence boundaries (regex `(?<=[.!?])\s+`).
3. Accumulate sentences into chunks up to the limit.
4. If a sentence exceeds the limit: split at word boundaries.
5. If a word exceeds the limit: split at character boundaries.

Whitespace between sentences is normalized to a single space. This function is provider-agnostic and used by both ElevenLabs (5 000–40 000 char models) and OpenAI (4 096 char hard limit).

## 5.2 Segment Processing

Multi-segment requests (via the `segments` parameter) are processed sequentially:

1. Each segment resolves its own voice and language independently, with per-segment overrides falling back to top-level defaults.
2. Per-segment `vibe_tags` override the top-level default (only for providers that support expressive tags).
3. Segments are synthesized to individual temp files.
4. `stitch_audio()` concatenates all segment files with `pause_ms` silence between each (default 500 ms).
5. 150 ms trailing silence is appended to prevent MP3 frame truncation artifacts.

## 5.3 Audio Stitching

`stitch_audio(segments, output_path, pause_ms)` uses pydub's `AudioSegment`:

- Loads each MP3 segment file.
- Inserts `AudioSegment.silent(duration=pause_ms)` between segments.
- Appends 150 ms trailing silence.
- Exports as MP3 to `output_path`.
- Raises `FileNotFoundError` if any segment is missing; `ValueError` if the segment list is empty.

pydub delegates to ffmpeg for MP3 decode/encode. This is the only place in the system that requires ffmpeg at runtime (playback uses `afplay` or `ffplay`, but stitching uses ffmpeg via pydub).

## 5.4 Playback Queue

Playback is serialized across all concurrent callers via POSIX file locking:

| Component | Details |
|---|---|
| Lock file | `~/.punt-labs/vox/playback.lock` |
| Lock type | `fcntl.flock(fd, LOCK_EX)` (blocking) |
| Timeout | 120 seconds |
| Player (macOS) | `afplay` |
| Player (other) | `ffplay -nodisp -autoexit -loglevel quiet` |
| Pending dir | `~/.punt-labs/vox/pending/` |

`enqueue(path)` copies the audio file to the pending directory (preventing deletion races with ephemeral cleanup), then spawns a fully detached subprocess that acquires the lock, plays, and deletes the pending copy. The parent returns immediately.

**What this does not do:** there is no priority queue, no cancellation, no skip-ahead. If three audio files are enqueued, they play in FIFO order. There is no way to interrupt a playing file or flush the queue.

### 5.5 Ephemeral Mode

When `ephemeral=true` (the default for `unmute`), output files go to `.vox/` in the current working directory—the same directory that holds `config.md`. Before each synthesis, previous ephemeral files are cleaned up by the client (the MCP server or CLI deletes only `.mp3` files in `.vox/`, preserving `config.md` and any other non-MP3 files).

Persistent output (via `record` or `ephemeral=false`) goes to `~/vox-output/` or a user-specified directory.

### 5.6 Format Handling

All providers output MP3. Cloud providers (ElevenLabs, OpenAI, Polly) produce MP3 natively. The `say` provider produces AIFF via the macOS `say` command, then converts to MP3 via ffmpeg. The `espeak` provider writes WAV via `espeak-ng`, then converts to MP3 via ffmpeg. This means all providers produce files that pydub can decode, and multi-segment stitching works across all backends when ffmpeg is available.

## 6 CLI and Config

### 6.1 CLI Commands

| Command | Purpose |
|---|---|
| `vox unmute [TEXT]` | Synthesize and play. `--from FILE` for batch input (JSON array). Ephemeral output to `.vox/`. |
| `vox record [TEXT]` | Synthesize and save. `--output PATH` or `--output-dir DIR`. No playback. |
| `vox vibe MOOD` | Set session mood. Special values: `auto` (infer from signals), `off` (disable). |
| `vox notify [y\|n\|c]` | Set notification mode. `y` = on-demand, `n` = off, `c` = continuous. Optional `--voice`. |
| `vox speak [y\|n]` | `y` = voiced, `n` = chimes only. |
| `vox voice NAME` | Set session voice. |
| `vox status` | Show session state (provider, voice, mood, signals). |
| `vox doctor` | Health checks: Python version, player binary, provider API keys, provider-specific checks. |
| `vox version` | Print version string. |
| `vox mcp` | Start MCP server on stdio. |
| `vox hook <event>` | Dispatch hook handler (stop, post-bash, notification, pre-compact, user-prompt-submit, subagent-start, subagent-stop, session-end). |

Global flags: `--json` (JSON output), `--verbose` (debug logging), `--quiet` (suppress non-JSON output). `--verbose` and `--quiet` are mutually exclusive.

### 6.2 Config File Format

Session state is stored in `.vox/config.md` as YAML frontmatter:

```
---
notify: "y"
speak: "y"
voice: "matilda"
provider: "elevenlabs"
model: "eleven_flash_v2_5"
vibe_mode: "auto"
vibe: "focused coding session"
```

```
vibe_tags: "[calm] [focused]"
vibe_signals: "tests-pass@10:45,git-commit@10:47"
---
```

| Field | Values | Purpose |
|---|---|---|
| `notify` | `y`, `n`, `c` | Notification mode |
| `speak` | `y`, `n` | Voice vs. chime mode |
| `voice` | string or null | Session voice name |
| `provider` | string or null | TTS provider override |
| `model` | string or null | TTS model override |
| `vibe_mode` | `auto`, `manual`, `off` | Mood tracking mode |
| `vibe` | string or null | Human-readable mood |
| `vibe_tags` | string or null | ElevenLabs expressive tags |
| `vibe_signals` | string or null | Timestamped signal history |

`resolve_config_path()` uses `git rev-parse --git-common-dir` to locate the config file in worktree setups, ensuring all worktrees sharing a repository share the same session state. The result is cached via `functools.lru_cache(maxsize=1)`.

`write_field()` and `write_fields()` validate keys against an allowlist before writing. Unknown keys raise `ValueError`. If the config file does not exist, a minimal file is created with YAML frontmatter delimiters.

### 6.3 Credential Management

Vox does not store credentials. Provider API keys are read from environment variables:

| Variable | Provider |
|---|---|
| `ELEVENLABS_API_KEY` | ElevenLabs |
| `OPENAI_API_KEY` | OpenAI |
| AWS credential chain | Polly (via boto3) |

There is no credential file, no keychain integration, and no encrypted storage. `vox doctor` checks for the presence of these variables and reports their status.

## 7 Performance

### 7.1 Latency Characteristics

| Provider | Typical Latency | Notes |
|---|---:|---|
| ElevenLabs (flash) | 200–800 ms | Streaming API, buffered to disk |
| ElevenLabs (v3) | 500–2 000 ms | Higher quality, higher latency |
| OpenAI (tts-1) | 300–1 000 ms | No streaming, single HTTP call |
| OpenAI (tts-1-hd) | 500–1 500 ms | Higher quality variant |
| Polly (neural) | 200–600 ms | AWS region-dependent |
| macOS `say` | 50–200 ms | Local, no network |
| `espeak-ng` | 20–100 ms | Local, no network |

These are synthesis latencies (time to first byte from provider). Total time-to-audio includes stitching (10–50 ms for pydub operations) and playback queue wait (0 ms if queue is empty, unbounded if other audio is playing).

### 7.2 Caching

There is no intentional audio caching. Deterministic filenames (MD5 hash of text) mean that repeated calls for the same text produce the same output path. The MCP server's `_synthesize_segments()` skips synthesis when the output file already exists, which provides incidental caching for non-ephemeral

calls. In ephemeral mode, `clean_ephemeral()` deletes prior MP3 files before each call, so the skip rarely triggers.

Voice lists are cached globally per provider (populated on first call, never refreshed within a process). Config path resolution is cached via `lru_cache(maxsize=1)`.

### 7.3 Streaming vs. Batch

ElevenLabs returns a streaming response (chunk generator). Vox iterates the generator to completion and writes the full file before playback. This adds latency equal to the full synthesis time but simplifies the pipeline: every provider produces a complete file, and the playback system has a single code path.

**What this means:** for a 30-second audio clip, the user waits for the full 30 seconds of synthesis before hearing anything. True streaming playback (play while synthesizing) is not implemented.

## 8 Security

### 8.1 Threat Model

Vox runs in the user's process space with the user's credentials. The primary trust boundary is the MCP tool interface: Claude Code can call any tool, but tools only perform synthesis and configuration—no file deletion, no network access beyond provider APIs, no shell execution.

| Threat | Description | Mitigation |
|---|---|---|
| API key exposure | Provider keys in env vars visible to any process in the session. | Standard env var practice. No logging of key values. `doctor` reports presence, not content. |
| Unintended audio | Audio plays without user consent, e.g., during a meeting. | Default `notify=n`. No audio until explicit opt-in. `/mute` immediately silences. |
| Cost accumulation | Rapid synthesis calls exhaust provider quotas. | No rate limiting in Vox. Provider-side quotas are the only protection. |
| Pending file race | Concurrent processes write to `pending/` with colliding filenames. | Filenames include PID prefix. `flock` serializes playback. |

### 8.2 No Audio Until Opt-In

This is the system's primary safety property. On fresh install:

- `.vox/config.md` does not exist.
- `notify` defaults to `"n"`.
- All hook handlers check `notify` first and return immediately if `"n"`.
- The `SessionStart` hook deploys commands but produces no audio.
- No MCP tool produces audio unless the agent explicitly calls `unmute`.

The user must take an affirmative action (`/unmute`, `/vox y`, or the agent calling `notify(mode="y")`) to enable any audio output.

### 8.3 Credential Isolation

Each provider file is the only file that imports its SDK:

- `src/punt_vox/providers/elevenlabs.py` — only file importing `elevenlabs`

- `src/punt_vox/providers/openai.py` — only file importing `openai`
- `src/punt_vox/providers/polly.py` — only file importing `boto3`

API clients are instantiated inside provider constructors. No client objects are passed between modules. No credentials are logged at any level.

# 9 Platform Integration

## 9.1 macOS

- **Playback.** `afplay` is the primary player, available on all macOS installations. `ffplay` is the fallback (requires Homebrew ffmpeg).
- **Offline fallback.** The `say` provider uses the system's built-in speech synthesis. Voice names are macOS-specific (e.g., `samantha`, `alex`).
- **File locking.** `fcntl.flock()` works correctly on APFS and HFS+.
- **Temp directory.** `.envrc` sets `TMPDIR` to `.tmp/` in the project root, keeping temp files workspace-local.

## 9.2 Linux

- **Playback.** `ffplay` from the `ffmpeg` package. No `afplay` equivalent. `resolve_player()` raises `FileNotFoundError` if neither is available.
- **Offline fallback.** The `espeak` provider wraps `espeak-ng` (preferred) or `espeak`.
- **File locking.** `fcntl.flock()` works on ext4, btrfs, and most Linux filesystems. NFS mounts may not honor `flock`; this is not handled.
- **No headless mode.** Playback requires an audio device. Running in a headless container or SSH session without PulseAudio/ALSA will cause `ffplay` to fail. There is no detection or graceful degradation for this case.

# 10 Plugin and Hook Integration

Vox integrates with Claude Code through a plugin manifest (`plugin.json`) and a hook registration file (`hooks.json`). The plugin registers an MCP server (key: `mic`) and nine lifecycle hooks.

## 10.1 Hook Registration

| Event | Script | Behavior |
|---|---|---|
| `SessionStart` | `session-start.sh` | Deploy slash commands, clean retired commands, auto-allow MCP tools. No audio. |
| `PostToolUse` (MCP) | `suppress-output.sh` | Format MCP tool output as compact UI panel text. |
| `PostToolUse` (Bash) | `signal.sh` | Classify stdout, append signal to `vibe_signals`. |
| `Stop` | `notify.sh` | Block with speech phrase or play chime. Returns {`decision: "block"`} to pause Claude while audio plays. |
| `Notification` | `notify-permission.sh` | Play chime or speak permission/idle phrase (async). |

| Event | Script | Behavior |
|---|---|---|
| PreCompact | pre-compact.sh | Speak "grabbing a snack" phrase (continuous mode only, async). |
| UserPromptSubmit | acknowledge.sh | Speak acknowledgment phrase (continuous mode only, async). |
| SubagentStart/Stop | subagent.sh | Announce subagent spawn/completion (continuous mode only, async). |
| SessionEnd | farewell.sh | Speak farewell, clear vibe_signals (async). |

## 10.2 Mute/Unmute Lifecycle

Three notification states control audio behavior:

| State | Hooks Fire? | Audio? |
|---|---|---|
| notify=n | All hooks return immediately | No audio from hooks. Agent can still call unmute directly. |
| notify=y | Stop, Notification, SessionEnd | Chimes (speak=n) or speech (speak=y) on task completion and permission requests. |
| notify=c | All hooks fire | Continuous narration: acknowledgments, subagent announcements, pre-compact phrases, farewells. |

speak is orthogonal to notify:

- speak=y: hooks synthesize speech via provider.
- speak=n: hooks play pre-recorded chime MP3 files from assets/ (no provider API call).

## 10.3 Signal Classification

Signal classification uses a single canonical pattern table in hooks.py. The PostToolUse (Bash) hook calls classify_signal() directly. The session watcher in watcher.py delegates to the same function via classify_output(), passing exit_code=None. The eight regex-based signal patterns are:

| Signal | Detection Patterns |
|---|---|
| lint-fail | Found [0-9]+ error |
| lint-pass | All checks passed, 0 errors |
| tests-pass | [0-9]+ passed, tests? ok, ✓.*passed |
| tests-fail | FAILED, AssertionError, ERRORS?\b |
| merge-conflict | CONFLICT |
| git-push-ok | Everything up-to-date, ->.*main |
| git-commit | ^\[.+\] .+, ^create mode |
| pr-created | pull/[0-9]+, created pull request |

If no pattern matches but exit code is non-zero, the signal is cmd-fail. Signals are appended as {signal}@{HH:MM} to the comma-separated vibe_signals config field.

### 10.4 Mood Inference

On Stop hooks, accumulated signals are deterministically mapped to ElevenLabs expressive tags. No LLM call is involved:

| Signal Trajectory | Tags |
|---|---|
| Recovery (fail → pass) | `[relieved]` |
| Shipped, no failures | `[satisfied]` |
| Shipped after failures | `[relieved] [satisfied]` |
| Ended with failure, many fails | `[frustrated] [sighs]` |
| Many passes, no failures | `[excited]` |
| Some passes | `[calm]` |
| Default | `[calm]` |

This only affects audio when the provider supports expressive tags (`eleven_v3` model only). All other providers ignore the tags silently.

## 11 Known Limitations

### 11.1 No Streaming Playback

Audio is fully synthesized to disk before playback begins. For long texts, this means the user waits for the entire synthesis to complete. ElevenLabs returns a streaming response, but Vox buffers it entirely. True streaming playback (play the first chunk while subsequent chunks synthesize) is not implemented.

### 11.2 No Intentional Audio Caching

The MCP server skips synthesis when the output file already exists at the deterministic path, but ephemeral cleanup deletes prior files before most calls. There is no cache layer, no TTL, and no cache invalidation. For non-ephemeral use patterns with repetitive text, the file-exists check provides incidental reuse. For ephemeral calls (the common path), every synthesis hits the provider API.

### 11.3 No Provider Failover

If the selected provider fails (API error, rate limit, credentials expired), the error propagates to the caller. There is no automatic fallback to a lower-tier provider, no retry with exponential backoff, and no circuit breaker.

### 11.4 No Playback Cancellation

Once audio is enqueued, it plays to completion. There is no skip, pause, cancel, or flush-queue mechanism. If three long audio clips are enqueued, they play sequentially with no way to interrupt.

### 11.5 Background Failure Opacity

The `unmute` and `record` MCP tools return predicted results before synthesis completes. If synthesis fails in the background thread (provider error, network timeout, disk full), the error is logged to stderr but not reported to the agent. The agent has no mechanism to detect or recover from synthesis failures after the tool call returns. The session watcher's autonomous speech also fails silently (bare `except Exception` with logging).

### 11.6 Concurrent Config Writes

`write_fields()` in `config.py` performs a read-then-write with no file-level lock. The MCP server's background synthesis thread and `PostToolUse` hook handlers can write `vibe_signals` concurrently. Both paths do unguarded read-modify-write on the same file. The race window is narrow but real in continuous mode with fast bash commands. The playback lock (`playback.lock`) governs audio playback, not config writes.

### 11.7 NFS File Locking

The playback serialization relies on `fcntl.flock()`. On NFS mounts, `flock` may not provide mutual exclusion. If `~/.punt-labs/vox/` is on an NFS filesystem, concurrent playback from multiple machines could overlap.

### 11.8 No Headless Audio Detection

On Linux, if no audio output device is available (headless container, SSH without PulseAudio), `ffplay` fails silently or with an error. Vox does not detect this condition and does not warn the user.

### 11.9 Voice List Staleness

Provider voice lists are cached globally on first access and never refreshed within a process lifetime. If a voice is added or removed on the provider side, the change is invisible until the process restarts (next Claude Code session).

## 12 Test Architecture

| Test File | What It Tests |
|---|---|
| `test_types.py` | Domain type validation, `generate_filename()` determinism |
| `test_core.py` | Text splitting edge cases, `TTSClient` batch/pair synthesis, stitching |
| `test_config.py` | Config read/write, worktree resolution, field validation |
| `test_output.py` | Output path resolution, env var handling |
| `test_normalize.py` | Text normalization for speech |
| `test_cache.py` | MP3 quip cache, content addressing |
| `test_playback.py` | `flock` serialization, `enqueue()` subprocess lifecycle |
| `test_hooks.py` | Signal classification, mood inference, hook handler dispatch |
| `test_cli.py` | CLI command smoke tests |
| `test_server.py` | MCP tool behavior with mocked providers |
| `test_elevenlabs_provider.py` | Voice resolution, chunking, voice settings, health checks |
| `test_openai_provider.py` | Rate mapping, chunking at 4 096 chars |
| `test_polly_provider.py` | VoiceConfig bundling, engine preference, Polly API mocks |
| `test_say_provider.py` | macOS say command construction |
| `test_espeak_provider.py` | espeak-ng command construction |
| `test_client.py` | `VoxClient`/`VoxClientSync` WebSocket client for `voxd` |
| `test_service.py` | System-level daemon lifecycle (launchd, systemd) |

Quality gates: `ruff` (lint + format), `mypy` (strict), `pyright` (strict), `pytest`, `shellcheck` (all shell scripts in `hooks/` and `scripts/`). Zero violations required.

Polly test fixtures use `AudioSegment.silent(duration=50)` to generate valid MP3 bytes, because pydub delegates to ffmpeg which rejects fake byte sequences. Provider mocks use `side_effect=lambda` rather than `return_value` to produce fresh mock objects per call.